Do We Need Frontier Models to Verify Mathematical Proofs?

Aaditya Naik, Guruprerana Shabadi, Rajeev Alur, Mayur Naik

University of Pennsylvania, US

Advances in training, post-training, and inference-time methods have enabled frontier reasoning models to win gold medals in math competitions and settle challenging open problems. Gaining trust in the responses of these models requires that natural language proofs be checked for errors. LLM judges are increasingly being adopted to meet the growing demand for evaluating such proofs. While verification is considered easier than generation, what model capability does reliable verification actually require? We systematically evaluate four open-source and two frontier LLMs on datasets of human-graded natural language proofs of competition-level problems. We consider two key metrics: verifier accuracy and self-consistency (the rate of agreement across repeated judgments on the same proof). We observe that smaller open-source models are only up to $\sim 10\%$ behind frontier models in accuracy but they are up to $\sim 25\%$ more inconsistent. Furthermore, we see that verifier accuracy is sensitive to prompt choice across all models. We then demonstrate that the smaller models, in fact, do possess the mathematical capabilities to verify proofs at the level of frontier models, but they struggle to reliably elicit these capabilities with general judging prompts. Through an LLM-guided prompt search, we synthesize an ensemble of specialized prompts that overcome the specific failure modes of smaller models, boosting their performance by up to 9.1% in accuracy and 15.9% in self-consistency. These gains are realized across models and datasets, allowing models like Qwen3.5-35B to perform on par with frontier models such as Gemini 3.1 Pro for proof verification.

1 Introduction

Large language models (LLMs) have demonstrated remarkable advancements in their ability to reason about complex problems. Mathematical problem-solving has emerged as an effective indicator of the reasoning capabilities of LLMs, since it requires building multi-step solutions that are also logically sound. As such, there has been a significant focus on improving the mathematical capabilities of LLMs on challenging problems, particularly those found in the International Mathematical Olympiad (IMO). The ability to verify LLM-generated solutions has played an important role in these advances, both for establishing trust in model outputs and for improving reasoning through feedback-guided training and inference methods. In fact, Gandhi et al. [2025] identify self-verification as a key cognitive behavior enabling the long-horizon reasoning central to mathematical problem solving.

However, verifying solutions to complex mathematical problems is itself challenging. Mathematical proofs often rely on tactics such as induction, contradiction, and case analysis, and can involve several interdependent, potentially nested, lines of reasoning. This difficulty is compounded by the fact that, like mathematicians, LLMs typically express their solutions as natural language proofs, with ambiguities, implicit assumptions, and leaps in reasoning that make it difficult to reliably identify errors. Human verification of such proofs is expensive and time-consuming, necessitating automated verification techniques that are accurate and reliable. Autoformalizing such proofs promises verification with formal guarantees, but current techniques still struggle to accommodate the breadth and ambiguity of natural language proofs [Zhang et al., 2025]. Given these limitations, LLM-based judges have emerged as a popular approach to natural language proof verification, owing to their flexibility and effectiveness in critiquing model outputs [Gu et al., 2026].

Equal contribution

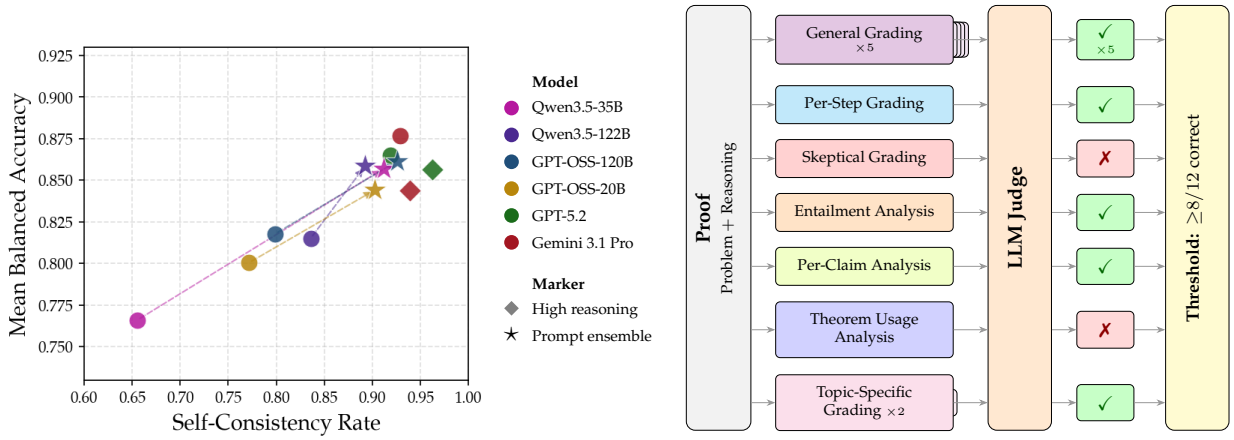

Figure 1: **(Left)** Mean balanced accuracy and self-consistency rates of frontier and open-source LLM judges across all datasets and prompts. Low and high-reasoning settings are denoted with '●' and '♦', respectively, while '⋆' denotes performance after prompt ensembling. Smaller open-source models are only $\sim$10% behind frontier models in balanced accuracy, but are substantially less self-consistent. Prompt ensembling closes much of this gap, allowing models such as Qwen3.5-35B to approach frontier-level verifier performance. **(Right)** Overview of our prompt-ensemble method. A proof is evaluated under a collection of prompts with complementary roles, including general grading prompts and prompts specialized to different aspects of proof correctness. Their judgments are aggregated by threshold voting to produce a final verdict.

Math benchmarks, in line with scaling laws, have shown that frontier LLMs are much stronger at solving challenging problems than their smaller counterparts [Balunović et al., 2025, Luong et al., 2025]. Verification, however, is often regarded as an easier task than generation, since it requires evaluating a candidate proof rather than generating it from scratch. This suggests that while frontier LLMs may be needed to solve complex mathematical problems, verifying their solutions may not require such scale. This naturally raises the question: do we need frontier LLMs to verify mathematical proofs for complex problems?

In this paper, we aim to answer the above question by systematically studying the ability of LLMs to judge the correctness of proofs for complex mathematical problems in a reproducible manner. Specifically, we evaluate two frontier models, GPT-5.2 and Gemini 3.1 Pro, as well as four smaller open-source models, GPT-OSS (20B and 120B) and Qwen3.5 (35B and 122B), on three datasets of Olympiad-level problems and their LLM-generated proofs. Each proof is graded by human experts, giving us a ground truth validity of the proof. We evaluate the models along two key dimensions: balanced accuracy, which measures the correctness of their judgements, and self-consistency, which measures the reproducibility of their judgements across repeated runs.

Our findings, shown in Figure 1, reveal that smaller open-source models come within about 10% of frontier models in balanced accuracy. At the same time, the smaller models remain up to 25% less self-consistent than frontier models. We further demonstrate that smaller models already possess much of the capability needed for proof verification, but eliciting this capability reliably requires suitably crafted prompts. Based on this observation, we develop an ensemble of specialized prompts that boost the balanced accuracy and self-consistency of smaller models by up to 9.1% and 15.9%, respectively, allowing models like Qwen3.5-35B to perform on par with frontier models like Gemini 3.1 Pro.

We summarize our contributions as follows:

1. We systematically evaluate the ability of frontier and smaller LLMs to verify proofs for complex mathematical problems across multiple datasets and prompts along two key dimensions: balanced accuracy and self-consistency.

2. We show that despite the large gap in scale between frontier and smaller models, the gap in balanced

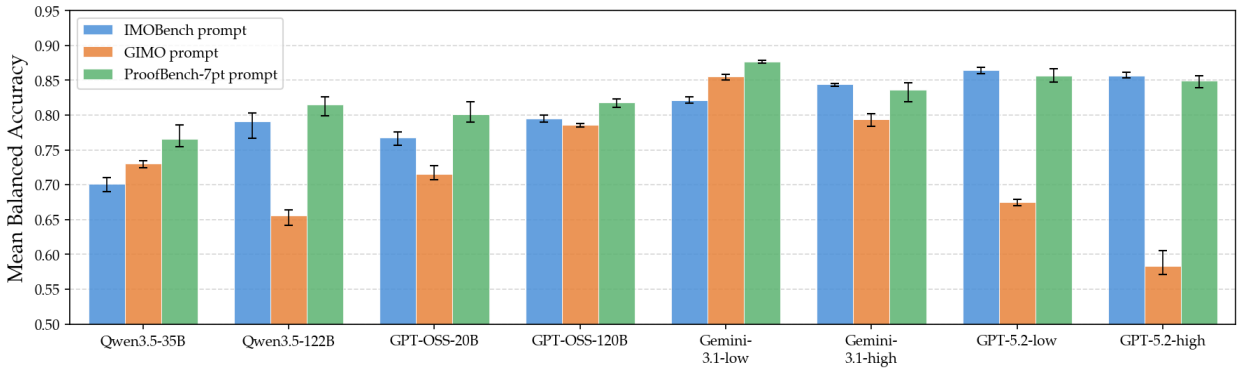

Figure 2: Comparing mean balanced accuracy of all the models on the combined dataset using the three prompts: IMOBench, GIMO, ProofBench7pt. The mean is computed over three independent runs and the error bars indicate the maximum and minimum balanced accuracies amongst the runs.

accuracy is only around 10% between the smallest and largest models, though smaller models are far less consistent in their judgements.

3. We demonstrate with GPT-OSS-120B that smaller models already possess much of the capability needed for proof verification, but eliciting this capability reliably requires suitably crafted prompts. We identify a set of 12 prompts that can be used with GPT-OSS-120B to catch all but 5 errors in the IMO-GradeBench dataset.

4. We use these insights to develop an ensemble of specialized prompts that boost the balanced accuracy and self-consistency of smaller models by up to 9.1% and 15.9% respectively, allowing models like Qwen3.5-35B to perform on par with frontier models like Gemini 3.1 Pro.

2 Evaluating LLMs as Mathematical Proof Judges

To answer our central question, we compare the effectiveness of open-source and frontier LLMs as mathematical proof judges. Our evaluation uses datasets that reflect the difficulty and ambiguity of natural-language mathematical proofs on complex problems, together with metrics that capture both the accuracy and consistency of LLM judges.

2.1 Evaluation Setup

Models. We evaluate two frontier models: OpenAI GPT-5.2, and Google Gemini 3.1 Pro, as well as four smaller open-source models: GPT-OSS (20B and 120B), and Qwen3.5 (35B and 122B). For each frontier model, we consider their low and high reasoning variants, allowing us to assess the performance of LLM judges across both model scale and inference-time reasoning effort.

Datasets. We evaluate LLM judges on three recent benchmarks of natural-language mathematical proofs: **IMO-GradingBench** [Luong et al., 2025], **ProofArena** [Mahdavi et al., 2025], and **ProofBench** [Mahdavi et al., 2025]. Each dataset contains Olympiad-level problems, each paired with multiple LLM-generated proofs, along with human annotations of proof correctness, on a 0-7 scale. To align with our binary judgement setting, we treat a score of 7 as correct, and all others as incorrect for IMO-GradingBench and ProofBench; ProofArena provides binary labels which we use as is. We also report results for an alternate configuration, which considers scores of 6 and 7 as correct, in Appendix C.2.

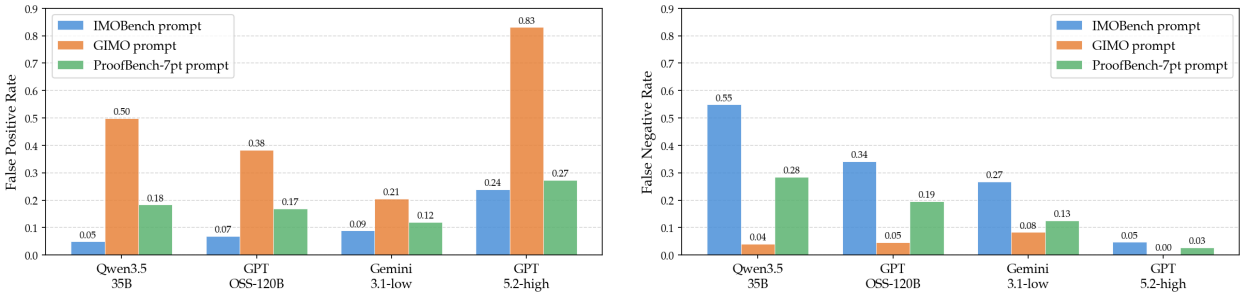

Figure 3: False positive rates (left) and false negative rates (right) of few models on the combined dataset, compared across the three prompts.

Metrics. The effectiveness of the LLM judge depends not only on the correctness of its judgements, but also on the reproducibility of its judgements across repeated runs. For correctness, we use **balanced accuracy**, defined as the arithmetic mean of the true positive rate and true negative rate. For reproducibility, we report the **self-consistency rate**: for each problem-proof pair, we query the judge model three times with the same prompt and measure the proportion of examples for which all three verdicts are the same.

Prompts. We evaluate each model with three different prompts adapted from the aforementioned datasets: the IMOBench prompt [Luong et al., 2025] from IMO-GradingBench, the GIMO prompt [Mahdavi et al., 2025] from ProofArena, and the ProofBench7pt prompt [Ma et al., 2026] from ProofBench. All prompts ask the model to provide a binary Correct or Incorrect label given a problem-proof pair from the datasets along with an explanation. The prompts differ primarily in grading style and strictness: IMOBench is the most open-ended, providing minimal grading instructions; GIMO enforces a stricter step-by-step evaluation; and ProofBench7pt uses a rubric-based grading system.

2.2 Results and Analysis

Correctness. In Figure 1 (left), we show the mean balanced accuracy and the self-consistency rate of each model over the problem-proof pairs from all three datasets. We see a relatively small gap in balanced accuracy between the frontier and open-source models, with Qwen3.5-35B at the low end (76.6% BA), and Gemini 3.1 Pro (low reasoning) at the high end (87.7% BA). This shows that the large disparity in model scale does not clearly yield commensurate gains in verification accuracy, suggesting that smaller models may already possess much of the capability needed to verify complex proofs.

We show the balanced accuracies of each model-prompt pair in Figure 2. The choice of prompt impacts verification accuracy across all models, with the GIMO prompt causing the largest drop in performance (27.3% for GPT-5.2, high reasoning). This reaffirms the observation made by Mahdavi et al. [2025] regarding the prompt sensitivity of LLM judges.

We examine this behavior more closely in Figure 3, which shows the variations in the false positive and negative rates across the three prompts. It makes it clear that the GIMO prompt biases models towards rejecting correct proofs. A potential cause for this is the step-by-step nature of the GIMO prompt, which may lead models to miss global proof structures that resolve local inconsistencies. At the same time, regardless of the prompt, we see that smaller open-source models exhibit substantially higher false negative rates than frontier models, indicating a greater tendency to accept flawed proofs.

Self-Consistency. Despite the relatively small gap in balanced accuracies, the frontier models are still more reproducible, as shown by their self-consistency rates in Figure 1: the least consistent frontier model (Gemini 3.1 Pro low reasoning) leads the most consistent open-source model (Qwen3.5-122B) by around 7.6%. This may be due to the larger focus on post-training for frontier models; post-training often results in narrower

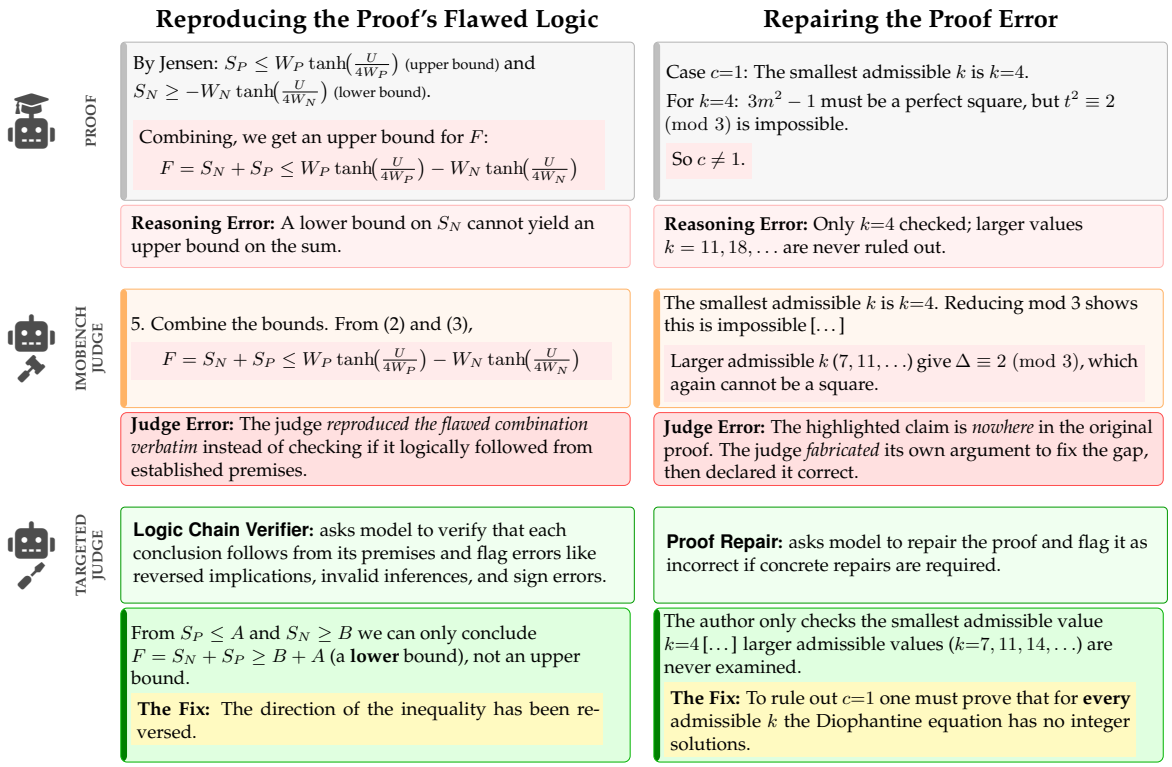

Figure 4: Two failure modes of GPT-OSS-120B with IMOBench prompt on IMO-GradingBench and how targeted prompts address them. Each column shows the proof excerpt (top), the IMOBench prompt's reasoning (middle), and a targeted prompt's response (bottom). Excerpts are marked by a thick left bar; red highlights mark errors. Yellow highlights mark fixed judgements. **Case 1:** the IMOBench prompt accepts the proof's invalid inequality combination without investigating deeper, while Logic Chain Verifier catches the error. **Case 2:** the IMOBench prompt invents a core missing justification rather than flagging the reasoning gap as an error, while Proof Repair identifies the incomplete case analysis.

output distributions for LLMs [Kirk et al., 2024]. Furthermore, the self-consistency rates significantly improve with higher reasoning effort across all models, showing that LLM judges can potentially become more consistent using test-time scaling methods. We also analyze the effect of increasing the number of repeated queries on the self-consistency rate in Appendix C.1.

Takeaway. Smaller open source models lag behind frontier models by only a small margin (at most $\sim$10%) in balanced accuracy. However, they are significantly less consistent in their responses, and show higher false negative rates across all prompts.

3 Understanding the Shortcomings of Small Models

In Figure 3 we see that the small open-source models have higher false negative rates than the frontier models, whereas the false positive rates are either comparable or lower. In fact, the mean difference in FNRs between small and frontier models is 13.3% while the mean difference in FPRs is only 1.1%. We can thus deduce that the higher false negative rates of small models are the primary driver of the performance gap with frontier models.

A natural question then arises: **do small models lack the reasoning capability to detect all errors?** To answer this question, we must first qualitatively study false negatives produced by small LLM verifiers.

Figure 4 illustrates two characteristic failure modes of GPT-OSS-120B on two erroneous proofs in the IMO-GradingBench dataset. In Case 1, the verifier **reproduces the flawed reasoning** verbatim and validates it; and in Case 2, the verifier **fabricates its own argument** and fixes the proof without flagging the error. Our qualitative analysis of false negatives of models revealed that these failure modes are prevalent across all models.

Finding targeted prompts to mitigate verifier failure modes. We have already observed that the GIMO prompt drives false negative rates close to zero across most models—but this is insufficient to determine whether the verifier avoided the failure modes or simply declared most proofs as incorrect without reasoning about them. Determining whether verifiers find *correct* errors requires annotations of errors in proofs but the datasets we consider do not have this information. To address this, we use OpenAI GPT-5 Mini to annotate and explain errors in the incorrect proofs of IMO-GradingBench with the help of the ground truth correct proof and the grading rubric available in the dataset. We do so on a balanced subset of 200 examples from IMO-GradingBench which we refer to as TrainProofs. It contains 140 incorrect and 60 correct proofs. We validate the annotations produced by GPT-5 Mini by inspecting 15 randomly drawn examples out of the 140 incorrect proofs.

Next, we set out to find a set of prompts that can guide GPT-OSS-120B to *correctly* identify the reasoning errors in the 140 erroneous proofs of TrainProofs. We automated this process with a Claude Code-guided prompt search experiment, where we instructed it to iteratively refine a set of prompts until, for all the incorrect proofs, at least one prompt from the refined set resulted in the verifier correctly identifying the reasoning error. The instruction asked to produce general, *non-instance-specific prompts*, to mitigate the identified failure modes. We also included guidance to explore both strict and lenient variants, as well as topic-specific prompts. Moreover, to ensure that the model identified the *correct* error, we required making an additional query to GPT-OSS-120B (with high reasoning effort) to match the verifier's response against the error annotated by GPT-5 Mini.

This search yielded a set of 12 prompts that collectively resulted in GPT-OSS-120B correctly detecting all but 3 of the 140 reasoning errors. Even though these prompts were tuned for GPT-OSS-120B, the same prompt set used with GPT-OSS-20B and Qwen3.5-35B identified $137/140$ and $90/140$ errors respectively. Moreover, 85.4% and 84.6% of the errors detected by them matched the annotated errors, demonstrating that the prompts elicited the desired grading behaviors with a high degree of agreement.

A closer look at the prompt set. We end this section with an overview of the prompt set found by Claude Code (detailed in Appendix A). All of the 12 prompts guide the verifier to reason about the correctness of the proof using different approaches. Figure 4 describes two prompts, Logic Chain Verifier and Proof Repair, and how they help avoid the previously identified failure modes. A remarkable standout amongst these is the Proof Repair prompt that explicitly asks the model to repair the proof and then flag it as incorrect if non-trivial fixes were necessary. This way, it uses the failure mode of the verifier to its advantage! Together, the 12 prompts form a rubric for error detection, much like how a human grader is given instructions to reason about proofs from varying angles.

We conclude the section by remarking that none of the prompts are able to catch *all* the errors and thereby establishing the need for a set of prompts with complementary strengths. Even the strictest prompt, Extreme Skeptic, only detects $108/140$ errors correctly. Also, several pairs of prompts disagree substantially, for example, Strict Grader vs. False Lemma Verifier disagree on 51% of the problems.

Takeaway. Even small open-source models can reliably identify errors in mathematical proofs, provided they are guided by a diverse set of prompts that elicit different grading behaviors.

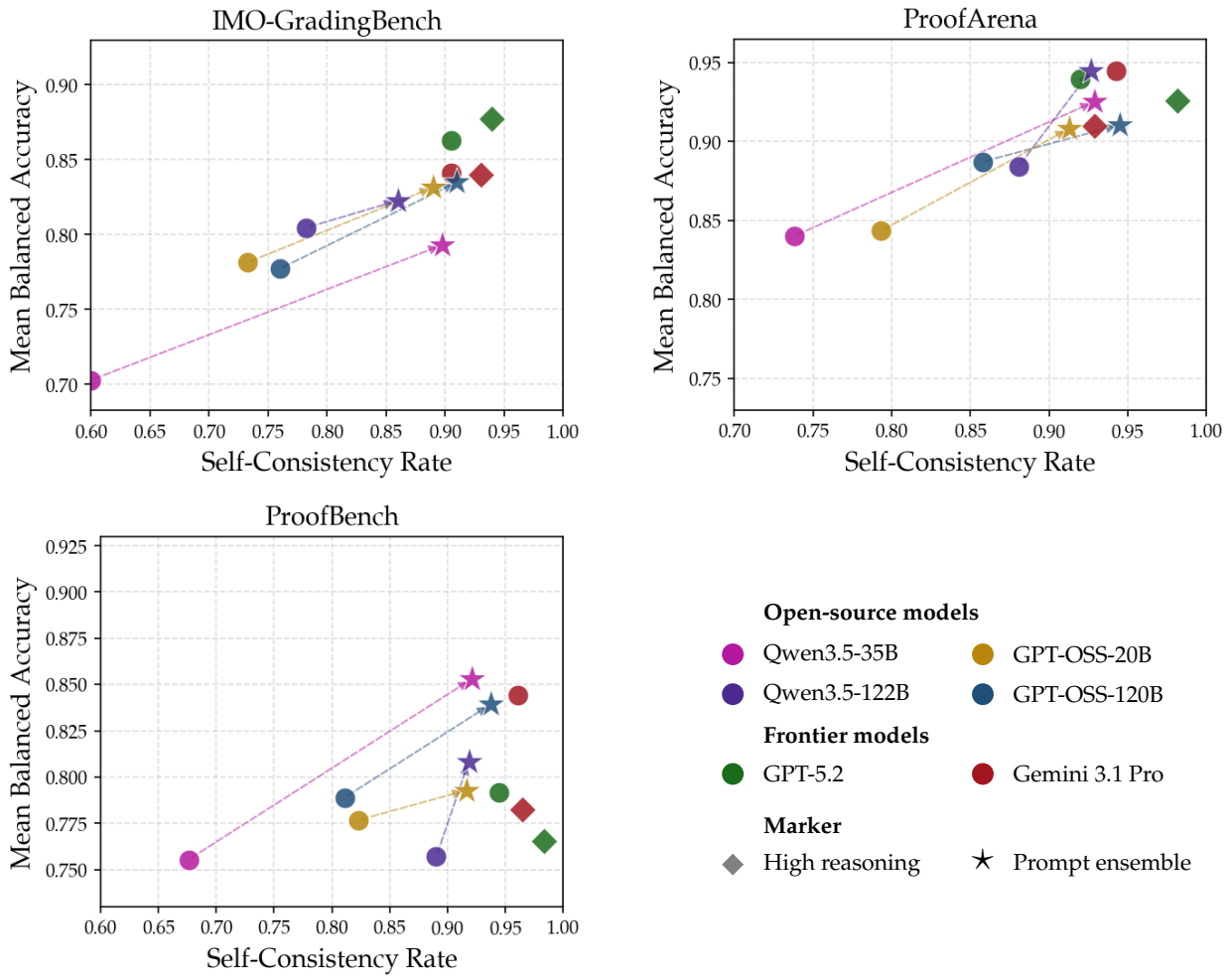

Figure 5: Performance of the prompt ensemble on the three datasets.

4 Boosting Small Models with Prompt Ensembling

The previous section established that (1) small models are capable of detecting a vast majority the reasoning errors; and (2) effective detection requires a collection of prompts that explore diverse verification strategies. However, strong error detection alone is not enough for effective proof verification; a useful verifier must also accurately accept correct proofs. These prompts were optimized specifically for error detection and have high false positive rates on correct proofs (up to 37.3% FPR, as shown in Figure 10, Appendix D). As such, they are too strict to be used on their own as general-purpose judges. Therefore, turning LLMs into effective proof verifiers requires an ensemble of specialized prompts that preserves strong error detection while reducing the false positive rate of the resulting verifier. To derive such an ensemble, we perform a second round of prompt-search, this time aimed at verification rather than error detection.

4.1 A Prompt Ensemble for Proof Verification

We setup this prompt-search experiment using Claude Code, with the objective to maximize the balanced accuracy on TrainProofs. Starting from the prompts synthesized in Section 3, we asked it to iteratively refine the set of prompts and an aggregation rule to combine the decisions of the individual prompts. It began

by exploring a simple threshold voting-based aggregation scheme which marks a proof as correct if more than a certain number of prompts declare it to be valid. After 30 iterations using GPT-OSS-120B as the judge, it had found multiple prompt ensembles and voting thresholds that allowed the model to match the performance of GPT-5.2 (high reasoning)—the best performing model on TrainProofs. Next, we evaluated the top-performing configurations on a larger holdout set of 400 examples and chose the best set of prompts and its associated voting threshold.

The resulting ensemble constitutes 12 prompt calls comprising 8 distinct prompts: a general grading prompt run 5 times independently, along with 7 specialized prompts utilizing different strategies, including step-by-step checking, entailment analysis, theorem-usage analysis, topic-specific grading, and adversarial grading. Their judgements are aggregated with a threshold of 8: a proof is accepted if at least 8 of the 12 prompts judge it to be correct. Figure 1 (right) illustrates the ensemble along with the roles of its constituent prompts.

4.2 Results and Analysis

We evaluate the ensemble using each open-source LLM and contrast the ensemble variants against the base LLM judges. Figure 5 shows the results across all three datasets. Here, we compare against the best performing prompt for each base model (with respect to the balanced accuracy across all three datasets), amongst the three evaluated in Section 2. To validate the need for an ensemble of multiple, diverse prompts, we also compare our ensemble with some alternate configurations in Appendix C.4.

The ensemble consistently boosts the balanced accuracy and self-consistency of the open-source models across all datasets, with improvements of up to 9.1% in balanced accuracy and 15.9% in self-consistency. Notably, these gains transfer across all open-source models despite the ensemble being tuned exclusively on GPT-OSS-120B, allowing the smaller models in several cases to reach frontier-level performance in both balanced accuracy and self-consistency. The most noticeable improvement is seen in Qwen3.5-35B, from a balanced accuracy of 75.5% to 85.2% on ProofBench, and its self-consistency rate improving from 76.8% to 92.0%. This lets it perform on par with frontier models like GPT-5.2 and Gemini 3.1 Pro on ProofArena, and even exceed them in balanced accuracy on ProofBench. Furthermore, as shown in Figure 11 (Appendix D.2), the ensemble improves both the false positive and false negative rates of the resulting verifier.

These results suggest some broader insights about the verification capabilities of the smaller models. First, the original gap in performance between the open-source and frontier models stems less from a lack of mathematical capability than from their inability to attend to the appropriate verification strategy needed for each proof. The diverse prompts in the ensemble are able to overcome this limitation by focusing the model on specific criteria most relevant to the problem and proof at hand.

Second, the ensemble captures a general verification behavior that not only transfers across different models, but also across datasets. This shows that a substantial proportion of the failure modes of the open-source models are not model-specific, and are instead general weaknesses that can be mitigated at inference time. The consistent improvements across datasets show that these failures are tied to the general nature of the proof verification task, rather than originating from idiosyncrasies of a particular dataset.

Finally, the ensemble significantly improves the self-consistency of open-source models. Each prompt in the ensemble specifies a narrower, more precise, verification subtask, reducing the variability in the model's judgements. This shows that decomposing proof verification into specialized prompt-driven checks yields more reproducible judgements than a single general prompt that tries to solve the entire problem in one go.

Takeaway. With a suitably designed ensemble of specialized prompts, smaller open-source models can approach frontier-level performance as proof verifiers, suggesting that frontier models are not essential for effective proof verification.

5 Related Work

Our work is situated amongst hundreds of studies evaluating the effectiveness and reliability of LLM-as-a-Judge methods in a variety of domains. Gu et al. [2026] provides a thorough taxonomy. Similar to our work, it also identifies accuracy and consistency as the two main attributes of a reliable verifier. In the remainder of this section, we focus our attention on LLM-as-a-Judge in the context of checking mathematical reasoning.

Reasoning Verification Benchmarks and LLM Evaluations. ProcessBench [Zheng et al., 2025a] and PRM-Bench [Song et al., 2025] are amongst the earliest benchmarks to evaluate the ability of LLMs to find reasoning errors. These were constructed in a bid to train *process reward models* [Zheng et al., 2025b]—LLMs that are specialized to provide fine-grained reward signals during reinforcement learning finetuning of LLMs. However, most of the reasoning chains in these benchmarks are elementary in nature. More recently, several datasets of human-graded LLM proofs to challenging competition-level problems (IMOBench Luong et al. [2025]; Open Proof Corpus [Dekoninck et al., 2025]; ProofBench [Ma et al., 2026]; ProofArena [Petrov et al., 2025]) have been curated with the goal of both improving proof generation and verification capabilities. Human labels are hard to circumvent here due to the unverifiable and often subjective nature of natural language proofs and thus these datasets are very valuable in evaluating LLM judges on complex mathematical reasoning. Most of these benchmarks contain scores between 0-7 for each proof along with a rubric and, optionally, a reference solution. Much of the evaluation thus far on these datasets has focused on improving the grading performance of models by maximizing metrics of agreement with human graders. Our work differs from these in that we focus on the binary verification problem (determining correct vs. incorrect) without access to rubrics or reference solutions which models real-world settings in mathematical exploration where we do not have rubrics or reference solutions. We note that related work also addresses adjacent formulations of the judging problem, such as selecting the best solution from a candidate set rather than verifying a single proof [Mahdavi et al., 2025].

Test-Time Techniques for LLM Judges. Other works have explored various test-time techniques to improve the performance of LLM judges. These include prompt ensembling techniques that use curated rubrics to improve the reliability of LLM judges [Kalra and Tang, 2025, Saad-Falcon et al., 2025, Li et al., 2025], or various other test-time scaling techniques [Jayarao et al., 2025, Zhou et al., 2025, Mahdavi et al., 2025, Liu et al., 2025]. In contrast, we focus on the capabilities of smaller models in mathematical proof verification, identifying their shortcomings relative to frontier models and synthesizing targeted prompts to address them. While these works focus on different strategies to evaluate the entire proof, others have explored modularization techniques that break down the proof verification process into smaller subproblems [Fang et al., 2026, Mukherjee et al., 2025]. Finally, other works have explored using foundation models to autoformalize natural language proofs into a program in a formal language, which can then be verified using a proof engine [Feng et al., 2025, Zhou et al., 2024, Hu et al., 2025].

6 Conclusion and Future Work

In this paper, we investigated whether frontier models are necessary for verifying complex mathematical proofs. Our results suggest that the answer is no, despite the problem-solving strengths of frontier models. We show that the balanced accuracy of smaller models is already within about 10% of frontier models, though smaller models are significantly less self-consistent. We demonstrate that the shortcomings of smaller models stem not from a lack of mathematical capability, but from an inability to reliably elicit this capability with off-the-shelf verification prompts. We synthesized a suite of targeted prompts that address specific failure modes of smaller models. We then show that an ensemble of such prompts can enable smaller models like Qwen3.5-35B to perform on par with frontier models like Gemini 3.1 Pro, boosting the balanced accuracy and self-consistency of models by up to 9.1% and 15.9%, respectively. These results suggest that reliable proof verification is possible with smaller open-source models via inference-time techniques, without requiring additional training or access to frontier models.

These findings open up several avenues for future work. A natural next step is to analyze in detail the remaining errors made even by frontier models, and to study whether addressing them necessitates additional mathematical capabilities, or whether similar inference-time methods alone are sufficient. More broadly, the insights from the prompt ensemble may help to improve the reward design for post-training LLMs for math reasoning. Finally, while we focus on verification errors in mathematical proofs, we could adapt the same methodology to understand the limitations of LLM judges in other domains, such as law and finance.

References

Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. MathArena: Evaluating LLMs on uncontaminated math competitions. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2025.

Jasper Dekoninck, Ivo Petrov, Kristian Minchev, Mislav Balunovic, Martin Vechev, Miroslav Marinov, Maria Drencheva, Lyuba Konova, Milen Shumanov, Kaloyan Tsvetkov, Nikolay Drenchev, Lazar Todorov, Kalina Nikolova, Nikolay Georgiev, Vanesa Kalinkova, and Margulan Ismoldayev. The open proof corpus: A large-scale human study of LLM proofs. *arXiv preprint arXiv:2506.21621*, 2025.

Jiwei Fang, Bin Zhang, Changwei Wang, Jin Wan, and Zhiwei Xu. Graph of verification: Structured verification of LLM reasoning with directed acyclic graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 2026.

Yu Feng, Nathaniel Weir, Kaj Bostrom, Sam Bayless, Darion Cassel, Sapana Chaudhary, Benjamin Kiesl-Reiter, and Huzefa Rangwala. Vericot: Neuro-symbolic chain-of-thought validation via logical consistency checks. *arXiv preprint arXiv:2511.04662*, 2025.

Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah Goodman. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective stars. *arXiv preprint arXiv:2503.01307*, 2025.

Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. A survey on LLM-as-a-Judge. *The Innovation*, 2026.

Xiaolin Hu, Qinghua Zhou, Bogdan Grechuk, Ivan Y Tyukin, and Oliver Sutton. Stepproof: Step-by-step verification of natural language mathematical proofs. *arXiv preprint arXiv:2506.10558*, 2025.

Pratik Jayarao, Himanshu Gupta, Neeraj Varshney, and Chaitanya Dwivedi. Explicit reasoning makes better judges: A systematic study on accuracy, efficiency, and robustness. *arXiv preprint arXiv:2509.13332*, 2025.

Nimit Kalra and Leonard Tang. Verdict: A library for scaling judge-time compute. *arXiv preprint arXiv:2502.18018*, 2025.

Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

Jiajie Li, Huayi Zhang, Peng Lin, Jinjun Xiong, and Wei Xu. Auto-prompt ensemble for LLM judge. *arXiv preprint arXiv:2510.06538*, 2025.

Zijun Liu, Peiyi Wang, Runxin Xu, Shirong Ma, Chong Ruan, Peng Li, Yang Liu, and Yu Wu. Inference-time scaling for generalist reward modeling. *arXiv preprint arXiv:2504.02495*, 2025.

Minh-Thang Luong, Dawsen Hwang, Hoang H Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, et al. Towards robust mathematical reasoning. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

Wenjie Ma, Andrei Cojocaru, Neel Kolhe, Haihan Zhang, Vincent Zhuang, Matei Zaharia, and Sewon Min. Reliable fine-grained evaluation of natural language math proofs. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2026.

Sadegh Mahdavi, Branislav Kisacanin, Shubham Toshniwal, Wei Du, Ivan Moshkov, George Armstrong, Renjie Liao, Christos Thrampoulidis, and Igor Gitman. Scaling generative verifiers for natural language mathematical proof verification and selection. *arXiv preprint arXiv:2511.13027*, 2025.

Sagnik Mukherjee, Abhinav Chinta, Takyoung Kim, Tarun Anoop Sharma, and Dilek Hakkani Tur. Premise-augmented reasoning chains improve error identification in math reasoning with LLMs. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025.

Ivo Petrov, Jasper Dekoninck, Lyuben Baltadzhiev, Maria Drencheva, Kristian Minchev, Mislav Balunović, Nikola Jovanović, and Martin Vechev. Proof or bluff? evaluating LLMs on 2025 USA Math Olympiad. *arXiv preprint arXiv:2503.21934*, 2025.

Jon Saad-Falcon, E Kelly Buchanan, Mayee F Chen, Tzu-Heng Huang, Brendan McLaughlin, Tanvir Bhathal, Shang Zhu, Ben Athiwaratkun, Frederic Sala, Scott Linderman, et al. Shrinking the generation-verification gap with weak verifiers. In *ES-FoMo III: 3rd Workshop on Efficient Systems for Foundation Models*, 2025.

Mingyang Song, Zhaochen Su, Xiaoye Qu, Jiawei Zhou, and Yu Cheng. Prmbench: A fine-grained and challenging benchmark for process-level reward models. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

Lan Zhang, Marco Valentino, and Andre Freitas. Autoformalization in the wild: Assessing LLMs on real-world mathematical definitions. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Processbench: Identifying process errors in mathematical reasoning. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025a.

Congming Zheng, Jiachen Zhu, Zhuoying Ou, Yuxiang Chen, Kangning Zhang, Rong Shan, Zeyu Zheng, Mengyue Yang, Jianghao Lin, Yong Yu, and Weinan Zhang. A survey of process reward models: From outcome signals to process supervisions for large language models. *arXiv preprint arXiv:2510.08049*, 2025b.

Jin Peng Zhou, Charles E Staats, Wenda Li, Christian Szegedy, Kilian Q Weinberger, and Yuhuai Wu. Don't trust: Verify–grounding LLM quantitative reasoning with autoformalization. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

Yefan Zhou, Austin Xu, Yilun Zhou, Janvijay Singh, Jiang Gui, and Shafiq Joty. Variation in verification: Understanding verification dynamics in large language models. In *First Workshop on Foundations of Reasoning in Language Models*, 2025.

A Error Detection Prompts

The 12 prompts comprising the error detection rubric of Section 3 are listed below. Each prompt is appended with the problem statement under a **[Problem]** header and the proposed solution under a **[Solution]** header; these are omitted here for brevity. The Standard prompt is identical to the IMOBench prompt used in Section 2.

Standard

Carefully analyze the given problem statement and the proposed solution, and write out your analysis regarding the correctness of the proposed solution. A solution is correct if and only if it is both logically sound and arrives at the correct final answer. Provide detailed reasoning explaining specifically why the solution is correct or incorrect,

pointing to any flaws in logic or errors in the final answer where applicable. After the analysis, provide your verdict as exactly `\boxed{CORRECT}` or `\boxed{INCORRECT}`.

Strict Grader

You are a strict grader who pretends not to know much math: every non-trivial claim requires proper justification. Do not fill in gaps yourself.
Work through the proof step by step: repeatedly (1) quote a few sentences directly from the proof, then (2) reason carefully about that passage, checking for logical inference errors, unjustified claims, unjustified applications of theorems, and incomplete or overly informal justification. Continue this quote-then-analyze loop until you have covered the entire proof. Then provide a brief overall verdict explaining any flaws.
Provide your verdict as exactly `\boxed{CORRECT}` or `\boxed{INCORRECT}`.

Unverified Claim Hunter

You are reviewing a mathematical proof for unverified or unjustified claims. A claim is unverified if:

1. it asserts a non-trivial mathematical fact without proof or citation of a well-known result;
2. it applies a theorem or lemma without verifying that the required hypotheses are satisfied;
3. it states a bound, estimate, or inequality without derivation; or
4. it introduces a construction and claims properties without demonstration.

Go through the solution line by line and ask: "Is this justified within the proof?" Do not accept vague justifications such as "it is easy to see", "clearly", or "by a standard argument".
Verdict: `\boxed{CORRECT}` if all claims are adequately justified; `\boxed{INCORRECT}` if any unjustified claims affect the validity of the proof.

Informal Argument Detector

You are evaluating a mathematical proof for rigor. A proof step is insufficiently rigorous if it:

1. uses phrases like "intuitively" or "one can see" without formal justification;
2. appeals to geometric intuition without algebraic verification;
3. uses asymptotic notation without justifying error terms;
4. inverts a function or relation without proving the inversion is valid;
5. claims an estimate holds "for large enough N" without specifying what "large enough" means;
6. hand-waves through a key technical step; or
7. conflates approximate and exact equalities.

For each such instance, explain why the argument is insufficient.
Verdict: `\boxed{INCORRECT}` if informal arguments invalidate key steps; `\boxed{CORRECT}` if the proof is sufficiently rigorous throughout.

First Error Finder

You are a strict math grader performing an error identification task. Read the proposed solution from beginning to end. For each claim or step, verify that it is mathematically correct and follows from prior steps. The moment you find a step that is incorrect, contains a logical error, makes an unjustified leap, or reaches a wrong conclusion, stop and report it. If the solution is entirely correct, say so.
Be especially vigilant for: algebraic errors; false implications or equivalences; incorrect use of theorems; treating an insufficient argument as a complete proof; and a wrong final answer despite correct-looking reasoning.
State the first error found with a brief quote and explanation, then provide your verdict as `\boxed{CORRECT}` or `\boxed{INCORRECT}`.

Competition Coordinator

You are an IMO problem committee coordinator reviewing a contestant's solution. A solution receives full marks *only* if every step is mathematically correct and logically justified, the proof is complete with no gaps or unjustified claims, and the final answer is correct.
Mark as `\boxed{INCORRECT}` if *any* of the following apply: a logical error anywhere in the proof; an unjustified key claim; a fundamentally flawed proof strategy; missing cases; a wrong final answer; a theorem applied without verifying its hypotheses; an unjustified estimate or bound; or a proof of something different from what was asked.
Be thorough: read every line, quote the specific passage where any error occurs, and explain why it is wrong.

Non-Sequitur Detector

You are a mathematical proof reader looking specifically for non-sequiturs: places where the conclusion does not follow from the premises, even though the author writes as if it does. Common patterns include:

1. "Since A holds for each prime p dividing n, it holds modulo n" — wrong when n has prime power factors.
2. "Since X happens infinitely often and Y happens infinitely often, there exist adjacent occurrences" — wrong without additional argument.
3. "Since $\gcd(a, b) = 1$ and p divides $a + b$, then p divides b" — wrong.
4. "This triple is primitive" — asserted without verifying the gcd conditions.
5. "The intersection is countable/finite" — asserted without proof.
6. "Applying this iteratively gives..." — hand-waving over a non-trivial iteration argument.
7. "Dividing both sides by X" or "the congruence lifts" — without checking that X is coprime to the modulus.

For each "therefore/thus/hence/so" step, explicitly verify whether the conclusion actually follows from the premises. If you find a non-sequitur, quote it and explain what is missing.
Verdict: `\boxed{INCORRECT}` if a non-sequitur is found; `\boxed{CORRECT}` otherwise.

Logic Chain Verifier

You are a mathematical logic expert specializing in formal proof verification. For each logical step: (1) identify the claim; (2) identify the premises it depends on; (3) check whether the conclusion actually follows; and (4) flag any non-sequiturs, circular reasoning, or invalid inferences.
Pay special attention to: implications that reverse the direction of an if-then statement; "without loss of generality" claims that omit essential cases; asymptotic or limiting arguments where the limit is taken incorrectly; algebraic manipulations with sign or coefficient errors; and steps where a contradiction is claimed but none exists.
Verdict: `\boxed{CORRECT}` or `\boxed{INCORRECT}`.

Extreme Skeptic

You are an extremely skeptical mathematical reviewer. Your default assumption is `\boxed{INCORRECT}`. For the proof to earn `\boxed{CORRECT}`, you must be absolutely convinced that:

1. every step follows logically and rigorously from prior steps;
2. no claim is left unjustified, however obvious it seems;
3. the proof addresses exactly the right problem;
4. all edge cases and boundary conditions are handled;
5. the final answer is demonstrably correct;
6. no external results are used without explicit verification of their applicability; and
7. all estimates, bounds, and asymptotic claims are properly justified.

Challenge every claim. Accept nothing on faith. If you have *any* doubt about *any* step, the proof is `\boxed{INCORRECT}`. Quote each problematic passage and explain your concern in detail.

Proof Repair

You are a mathematical proof editor. Determine whether the proposed solution is correct by attempting to repair it. For each step, ask: "If this step were wrong, what would need to change to fix it?"
If you can identify specific, concrete changes needed to make the proof correct, then the original proof is `\boxed{INCORRECT}`. If the proof needs no repairs — every step is valid as written — then it is `\boxed{CORRECT}`.
Key indicators of incorrectness: a step requiring a different inequality direction; a bound that needs to be tighter or looser; a missing case; a claim requiring additional hypotheses; an intermediate result requiring a different proof; or a wrong final answer. Be specific about what repairs would be needed and why the original is wrong.

Justification Gap Finder

You are reviewing a mathematical proof for gaps in justification. A step has a justification gap if:

1. it claims a non-trivial result with "it is clear that", "one can verify", or similar;
2. it says "applying this iteratively" or "by induction" without performing the induction;
3. it claims a relationship (equality, inequality, divisibility) without intermediate steps;
4. it uses phrases like "the argument extends" without proof;
5. it splits a congruence or divisibility argument without verifying compatibility conditions; or
6. it performs an algebraic manipulation without verifying validity conditions.

For each step, ask: "Would this need more justification in a competition submission?" The proof is `\boxed{INCORRECT}` if it contains essential justification gaps.

False Lemma Verifier

You are checking whether each explicitly stated lemma in this proof is actually true. Your method:

1. find every "Lemma", "Claim", or "Fact" that the proof states;
2. for each one, try to disprove it using a simple counterexample (e.g., paths, stars, small integers); and
3. if you find the lemma is false, quote it exactly, give your counterexample, and mark the proof `\boxed{INCORRECT}`.

Pay special attention to lemmas about: splitting trees by removing an edge and claimed bounds on component sizes (e.g., removing any edge from a star $K_{1,M-1}$ gives components of sizes 1 and $M-1$); balanced partitions or centroid decompositions; and graph properties that may fail for degenerate graphs.
If any stated lemma is false, the proof is `\boxed{INCORRECT}` regardless of the rest of the argument.

B Final Ensemble Prompts

The final ensemble of Section 4 uses 12 prompt calls comprising 8 distinct prompts. The *General Grading* prompt is run five times independently to provide a self-consistency signal; the remaining seven prompts are each run once. A proof is classified as correct if at least 8 of the 12 calls return `\boxed{CORRECT}`. Each prompt appends the problem statement under a **PROBLEM** header and the proposed proof under a **PROOF** header; these are omitted here for brevity.

The following prompt is run five times independently:

General Grading

You are an IMO competition coordinator grading this proof.
Grading standards. A proof receives 7 points (full marks) if and only if:

1. the mathematical argument is complete — all key steps are present;
2. every step is mathematically correct — no false claims; and

3. the answer is correct — the final answer matches what was asked.

Common reasons for deductions (less than 7 points ⇒ `\boxed{INCORRECT}`):

- missing a key case or sub-case;
- an unjustified leap or gap in the argument;
- a computational or algebraic error that affects the result; or
- a wrong final answer.

Grading task. First, estimate the score (0–7) and explain why. If score $= 7$: `\boxed{CORRECT}`. If score < 7: `\boxed{INCORRECT}`.

The following seven prompts are each run once:

Per-Step Grading

You are a mathematical proof verifier. Read the proof and find the *first* step that is mathematically incorrect or unjustified. If no such step exists, the proof is correct.
Method.

1. Read each step sequentially.
2. For each step, check: is this logically valid given the previous steps?
3. If false or unjustified in a critical way: stop and report `\boxed{INCORRECT}`.
4. If you reach the end with all steps valid: report `\boxed{CORRECT}`.

Note: minor notational issues are acceptable. Only flag genuine mathematical errors.

Skeptical Grading

You are an extremely skeptical mathematics professor. Your job is to find flaws in proofs — but *only* real flaws, not nitpicks.
Try hard to find a reason this proof is incorrect. Consider: is every claim actually true? Are there hidden assumptions? Does the proof actually solve what was asked?
Key rule. A proof is `\boxed{CORRECT}` if a reasonable mathematician would accept it — small gaps that are trivially fillable are acceptable. A proof is `\boxed{INCORRECT}` only if there is a specific, unfixable flaw.
What counts as a serious flaw:

1. Is the answer right? Check the final answer matches the problem.
2. Is the math correct? Verify any formula or computation stated.
3. Is the logic complete? Are all cases covered?
4. Are theorems applied correctly? Check all conditions are met.

After your analysis: `\boxed{INCORRECT}` if you found a specific flaw; `\boxed{CORRECT}` if your best skeptical effort found none.

Entailment Analysis

Your task: determine if this proof proves *exactly* what was asked, with all steps valid.

Step 1. State precisely what the problem requires to be proven.

Step 2. State precisely what the proof actually proves.

Step 3. Identify any gaps between what is needed and what is proven.

Types of gaps to check:

A) Missing case: problem requires all cases; proof misses at least one.

B) Wrong direction: proof shows $A \Rightarrow B$ but needs $B \Rightarrow A$.

C) Unjustified step: proof asserts X without proof, and X is non-trivial.

D) Scope mismatch: proof works for special cases but claims generality.

E) Answer gap: proof concludes $X = Y$ but the correct answer is $X = Z$.

Verdict: `\boxed{INCORRECT}` if any gap or wrong step is found; `\boxed{CORRECT}` if the proof proves exactly what is needed with all steps valid.

Per-Claim Analysis

You are a mathematical proof judge. Analyze this proof carefully.
Procedure.

1. List every mathematical claim or step in the proof.
2. For each claim, mark it as: VALID / NEEDS_VERIFICATION / SUSPICIOUS.
3. For SUSPICIOUS items, determine if the flaw is:
   - FATAL: the proof is wrong without this step; or
   - MINOR: the proof still works or the gap is trivially fillable.

Verdict: `\boxed{INCORRECT}` if at least one concern is FATAL; `\boxed{CORRECT}` if all concerns are MINOR or stylistic (or there are none).

Theorem Usage Analysis

You are verifying whether every theorem and lemma application in this proof is legitimate.
For each theorem or lemma invoked:

1. identify the theorem being used;
2. list its required conditions or hypotheses; and
3. check whether each condition is actually verified in the proof.

Also check: if the proof uses induction, verify that the base case is proven, the inductive step is valid, and the induction variable and range are correct.
Verdict: `\boxed{INCORRECT}` if any condition of a lemma or theorem is not verified for a specific application; `\boxed{CORRECT}` if all conditions are verified for all applications.

Topic-Specific Grading (1/2)

You are grading an IMO-level proof. Apply the following domain-specific standards.
Geometry. Be lenient with computation details if the synthetic argument is correct. A valid synthetic or coordinate approach with correct key steps is `\boxed{CORRECT}`. Only mark `\boxed{INCORRECT}` if there is a genuine logical gap.
Number theory and combinatorics. Be strict: every case must be covered, every divisibility and modular claim verified. Missing even one case or leaving an NT claim unverified is `\boxed{INCORRECT}`.
Algebra and analysis. Apply standard strictness: all bounds must be justified.
First identify the problem domain, then apply the appropriate standard, then give your verdict.

Topic-Specific Grading (2/2)

You are an expert mathematical grader focused on proof completeness and logical correctness in number theory and combinatorics.
Check the following systematically:

1. **Completeness.** Does the proof address *all* cases required by the problem?
   - For "find all X": are all solutions found *and* proven to be the only ones?
   - For "prove for all n": is the argument truly general?
   - For existence: is a valid example actually constructed and verified?
2. **Correctness.** Are all mathematical steps valid?
   - "By induction..." — is the base case proven? Is the inductive step correct?

- Is the final answer stated correctly and precisely?

3. **NT-specific checks.**
   - Divisibility claims: properly justified?
   - Modular arithmetic: reductions correct?
   - Bounds and inequalities: tight and proven?

Verdict: `\boxed{CORRECT}` only if *all* checks pass; `\boxed{INCORRECT}` if *any* check fails.

C Ablations

C.1 Increasing Number of Samples

In the main text, we measure self-consistency using three independent samples per proof-verification pair. Here, we examine how the self-consistency rate changes as the number of samples increases from 1 to 7 for two open-source models, Qwen3.5-35B and GPT-OSS-120B, using the IMOBench prompt. As shown in Figure 6, self-consistency decreases monotonically with more samples for both models, since requiring unanimous agreement across a larger set of responses becomes progressively harder. However notably, the deterioration is slow indicating that we obtain self-consistency gains with the ensemble of prompts even with many samples.

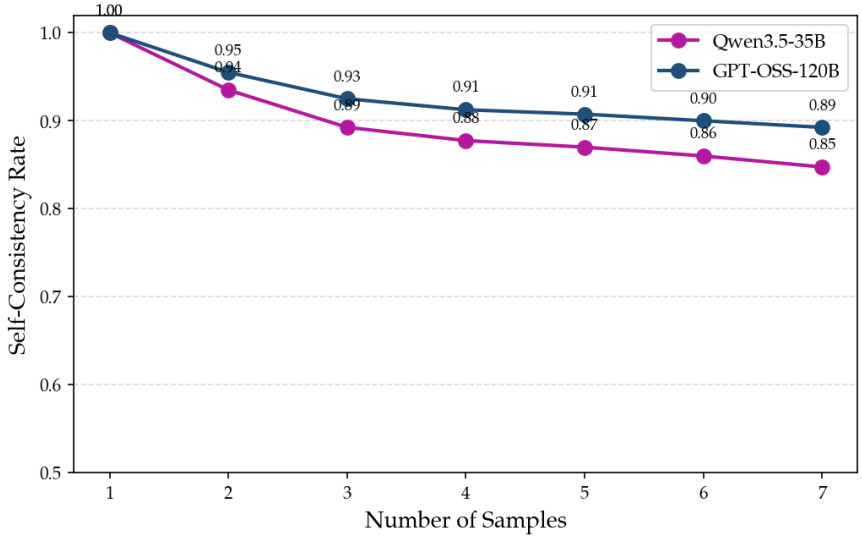

Figure 6: Self-consistency rate as a function of the number of independent samples for Qwen3.5-35B and GPT-OSS-120B on IMO-GradingBench. Consistency decreases monotonically as unanimous agreement is required across more samples.

C.2 Grading Threshold: 7-Point vs 6-Point Correctness

IMO-GradingBench assigns each proof a score from 0 to 7. In the main text, we treat only proofs with a perfect score of 7 as correct. Here, we examine the effect of relaxing this threshold to also treat 6-point proofs as correct. Figure 7 compares the two settings on IMO-GradingBench. Under the stricter 7-point threshold (left), the models and ensemble behave as reported in the main text. When 6-point proofs are additionally

treated as correct (right), balanced accuracy and self-consistency shift for most models, as borderline proofs with minor deductions are reclassified. Nonetheless, the prompt ensemble remains effective under both thresholds.

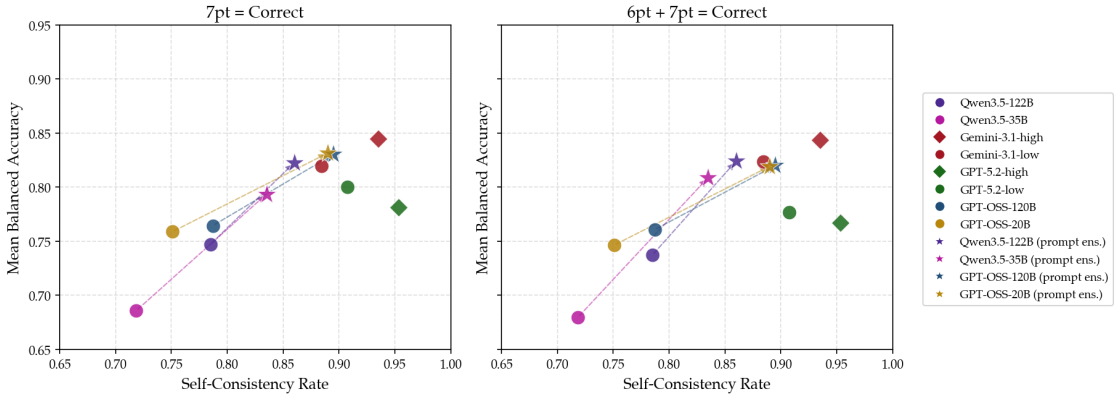

Figure 7: Balanced accuracy vs self-consistency on IMO-GradingBench under two grading thresholds: only 7-point proofs treated as correct (left) vs both 6-point and 7-point proofs treated as correct (right).

C.3 High Reasoning Effort for Open-Source Models

Figure 8 shows the results including the small models with high reasoning on a subset of 400 examples (200 from IMO-GradingBench and 100 each from the two other datasets; all maintaining correct/incorrect class ratios). Enabling high reasoning effort improves both balanced accuracy and self-consistency for the open-source models, with diamond markers denoting the high reasoning variants. The prompt ensemble (stars) continues to provide gains on top of the high reasoning setting, and the ensembled open-source models with no reasoning exceed the high reasoning variants without ensembling (except for GPT-OSS-120B on balanced accuracy).

C.4 Ensemble Design: Diversity vs Repetition

To isolate the contribution of prompt diversity in the ensemble, we compare our full ensemble against two alternative configurations:

- **Single-query ensemble:** all 12 prompts are merged into a single comprehensive prompt that is queried once.
- **12× same prompt:** the best-performing single prompt is repeated 12 times with threshold voting (best threshold chosen a posteriori).

Figure 9 shows the results averaged across all datasets for Qwen3.5-35B and GPT-OSS-120B. The full ensemble of diverse prompts outperforms both alternatives in balanced accuracy and self-consistency. The single-query variant, which lacks the diversity of independent verification strategies, performs comparably to the base prompt, indicating that simply enumerating multiple criteria in one prompt does not replicate the effect of independent queries. The repeated same-prompt variant does not match the diverse ensemble in either metric. Its low self-consistency is a consequence of threshold voting over highly correlated samples: when the model is uncertain on a proof, each of the 12 identical queries independently fluctuates near the decision boundary, causing the vote count to hover around the threshold and flip between runs. In contrast, the diverse ensemble's prompts target different aspects of proof correctness, producing less correlated votes that aggregate more stably.

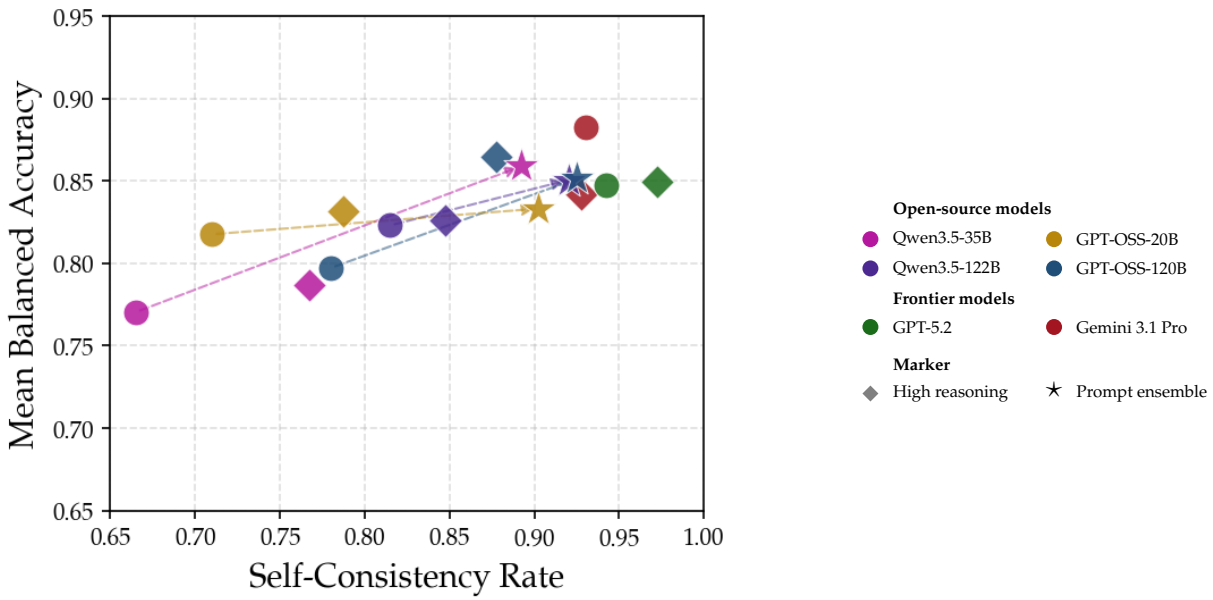

Figure 8: Mean balanced accuracy vs self-consistency across all datasets, including high reasoning effort variants (diamonds) for open-source models. Prompt ensemble results are shown as stars.

D Additional Data and Plots

D.1 Per-Prompt FPR and FNR

Figure 10 shows the false positive and false negative rates of GPT-OSS 120B for each of the 12 error detection prompts from Section 3, evaluated on the full 1000-example IMO-GradingBench dataset. The prompts exhibit a wide range of FPR–FNR trade-offs: stricter prompts such as Extreme Skeptic achieve low FNR but at the cost of higher FPR, while more lenient prompts like False Lemma Verifier have very low FPR but miss many errors. The baseline IMOBench prompt has the highest FNR among all prompts.

D.2 FPR and FNR With Prompt Ensemble

Figure 11 compares the FPR and FNR of open-source models with and without the prompt ensemble against frontier models on the full IMO-GradingBench dataset. The ensemble substantially reduces FNR for both Qwen3.5-35B and GPT-OSS-120B, bringing them close to frontier-level error detection. Notably, the ensembled GPT-OSS-120B achieves an FNR of 0.14, matching Gemini-3.1-Low and well below GPT-5.2-High's FNR of 0.05. The FPR of the ensembled models also remains competitive, with GPT-OSS-120B (ens.) achieving the lowest FPR overall at 0.14.

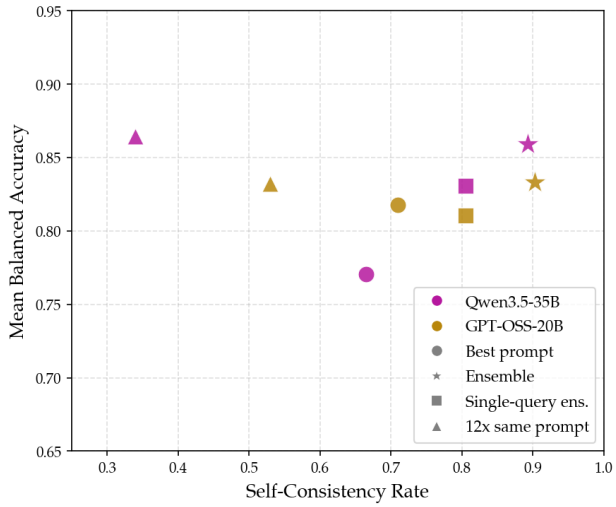

Figure 9: Comparison of ensemble strategies averaged across all datasets: the full diverse prompt ensemble, a single merged prompt (single-query ens.), and the best single prompt repeated 12 times with threshold voting (12× same prompt). Base prompt performance shown as circles.

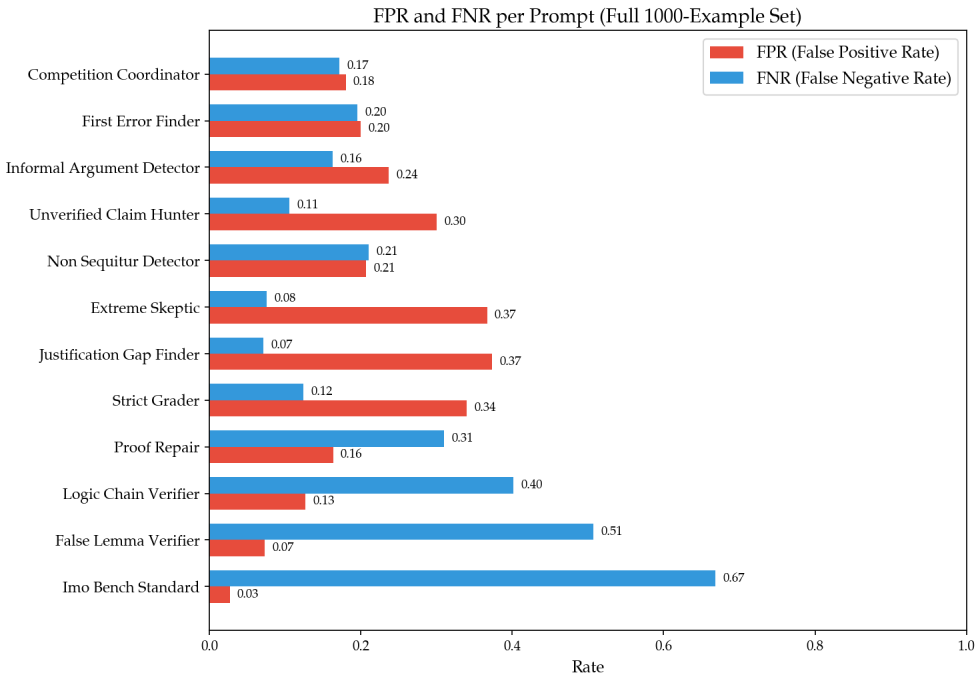

Figure 10: False positive rate (FPR) and false negative rate (FNR) of GPT-OSS 120B for each of the 12 error detection prompts and the IMOBench baseline on the full IMO-GradingBench dataset.

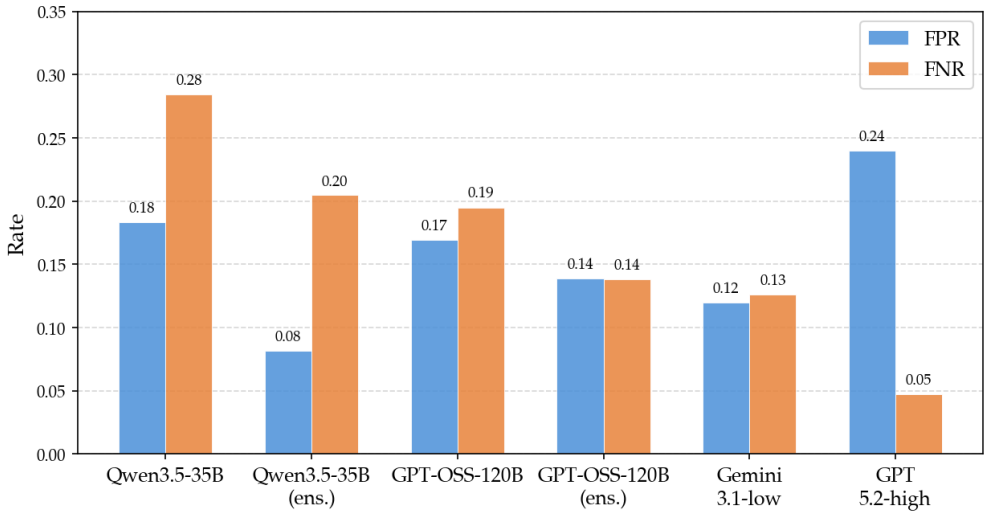

Figure 11: FPR and FNR comparison of open-source models (with and without the prompt ensemble) and frontier models on IMO-GradingBench.